

TP N°10 - Cassandra Parte I

Documentación oficial de Cassandra: <http://cassandra.apache.org/doc/latest/>

Instalación

En este caso vamos a utilizar dockers/contenedores:

- Instalar Docker Desktop for Windows o for Mac o Docker Engine en Linux
- Para descargar la versión oficial de Cassandra ejecutar: *docker pull cassandra*
- Crear una network para acceder a los puertos del contenedor sin exponer los del host: *docker network create network-cassandra*
- Levantar el contenedor con: *docker run --name Mycassandra --network network-cassandra -d cassandra* (Si van a usar DataGrip ver instrucciones más abajo)
- Para apagar el contenedor ejecutar: *docker stop Mycassandra*
- Para iniciarla nuevamente ejecutar: *docker start Mycassandra*
- Para listar todos los contenedores existentes ejecutar: *docker ps -a*

Para acceder a la línea de comandos dentro del contenedor: *docker exec -it Mycassandra bash*

Para acceder al cliente de Cassandra de línea de comandos, ejecuta *cqlsh*.

DataGrip: Si desean usar DataGrip, es necesario que expongan el puerto 9042 al momento de crear el contenedor: *docker run --name Mycassandra -p 9042:9042 -d cassandra*

El usuario para conectarse es **cassandra** y la password también **cassandra**.

A través del comando *show host* es posible validar el estado de la conexión.

Ejercicio 1:

Para efectos de la práctica, utilizaremos un servicio de música como ejemplo, que requiere de una Column Family canciones. Este servicio está compuesto por las columnas título, álbum, artista y una columna datos que contiene el archivo de audio real. La tabla utiliza una columna id del tipo UUID como clave primaria o clave de particionamiento.

- 1) Crear un keyspace con el siguiente nombre: *demo_cql_music*
cqlsh> create KEYSPACE demo_cql_music WITH replication = {'class': 'SimpleStrategy', 'replication_factor': '1'};

Usamos *SimpleStrategy* por ser una configuración simple, con una sola instancia de datos. *Replication_factor* indica el número de nodos que almacenan una copia de cada fragmento de datos, en este caso no tendremos réplicas.

- 2) Luego acceder al keyspace recién creado:
cqlsh> USE demo_cql_music;
- 3) Crear el Column Family canciones:
cqlsh:demo_cql_music> create TABLE canciones (id uuid PRIMARY KEY, titulo text, album text, artista text, data blob);

- 4) Ahora, insertar un registro en dicha tabla:

```
cqlsh:demo_cql_music> INSERT INTO canciones (id, titulo, album, artista) VALUES  
(7db1a490-5878-11e2-bcfd-0800200c9a66, 'Asilo', 'Salvavidas de Hielo', 'Jorge Drexler');
```

En una base de datos relacional, para simular una lista de reproducción generada por el usuario, se debería crear una tabla *lista_reproduccion* con una clave foránea a la tabla *canciones*. Sin embargo, en Cassandra se deben desnormalizar los datos porque los joins no son soportados.

- 5) Para representar los datos de una lista de reproducción, crear un Column Family:

```
cqlsh:demo_cql_music> create TABLE lista_reproduccion (id uuid, nro_cancion int, cancion_id  
uuid, titulo text, album text, artista text, PRIMARY KEY (id, nro_cancion));
```

- La combinación del *id* y *nro_cancion* identifican unívocamente a cada registro del Column Family *lista_reproduccion*. Se puede repetir el *id* de la lista de reproducción para varios registros, sin embargo los *nro_cancion* deben ser distintos
- *id* es la clave de particionamiento
- *nro_cancion* la clave de agrupamiento/clustering

- 6) Ahora, insertar un registro en dicha tabla:

```
cqlsh:demo_cql_music> INSERT INTO lista_reproduccion (id, nro_cancion, cancion_id, titulo,  
artista, album) VALUES (62c36092-82a1-3a00-93d1-46196ee77204, 1,  
7db1a490-5878-11e2-bcfd-0800200c9a66, 'Asilo', 'Jorge Drexler', 'Salvavidas de Hielo');
```

- *Nro_cancion* = 1 representaría la primera canción de la playlist

- 7) Insertar varios registros en el Column Family *canciones* utilizando la función *now()* para rellenar el campo *id*. Luego obtener los *id* generados, e insertar los nuevos registros en el Column Family *lista_reproduccion* en el orden que quiera

- 8) Ejecutar una consulta de ordenamiento por *nro_cancion*:

```
cqlsh:demo_cql_music> SELECT * FROM lista_reproduccion where id =  
62c36092-82a1-3a00-93d1-46196ee77204 ORDER BY nro_cancion DESC;
```

- 9) Ejecutar una consulta filtrada por artista:

```
cqlsh:demo_cql_music> SELECT * FROM lista_reproduccion WHERE artista='Jorge Drexler';
```

- Cassandra rechaza la consulta porque el campo *artista* no es una columna de la clave de partición

- 10) Una opción es utilizar la opción *ALLOW FILTERING*, ya que hemos insertado un conjunto pequeño de canciones en la lista de reproducción:

```
cqlsh:demo_cql_music> SELECT * FROM lista_reproduccion WHERE artista='Jorge Drexler'  
ALLOW FILTERING;
```

- La consulta anterior es bastante ineficiente, ya que realiza una búsqueda secuencial en todos los nodos que contienen datos de la tabla en cuestión

- 11) La otra es crear un índice para el campo artista, y así Cassandra podrá responder la consulta anterior de forma más eficiente:

```
cqlsh:demo_cql_music> CREATE INDEX indice_artista ON lista_reproduccion (artista);
```

- Esta opción es más eficiente que usar ALLOW FILTERING. No obstante, una búsqueda utilizando índices es más lenta que si el campo fuese parte de la clave de particionamiento (partition key)

- 12) En el servicio de música se quiere incluir la capacidad de etiquetar las canciones. Debido a que en el Column Family *lista_reproduccion* ya existe debemos utilizar el comando ALTER TABLE:

```
cqlsh:demo_cql_music> ALTER TABLE lista_reproduccion ADD etiquetas set <text>;
```

- 13) Es posible actualizar registros ya existentes. Probar añadiendo etiquetas a las listas de reproducción:

```
cqlsh:demo_cql_music> UPDATE lista_reproduccion SET etiquetas = etiquetas + {'2017'} WHERE id = 62c36092-82a1-3a00-93d1-46196ee77204 AND nro_cancion=1;
```

```
cqlsh:demo_cql_music> UPDATE lista_reproduccion SET etiquetas = etiquetas + {'guitarra'} WHERE id = 62c36092-82a1-3a00-93d1-46196ee77204 AND nro_cancion=1;
```

```
cqlsh:demo_cql_music> UPDATE lista_reproduccion SET etiquetas = etiquetas + {'Mon Laferte'} WHERE id = 62c36092-82a1-3a00-93d1-46196ee77204 AND nro_cancion=1;
```

- 14) Al igual que como se hizo para el campo artista, para poder hacer búsquedas sobre la colección de etiquetas se debe indexar la misma:

```
cqlsh:demo_cql_music> CREATE INDEX indice_etiquetas ON lista_reproduccion (etiquetas);
```

```
cqlsh:demo_cql_music> SELECT * FROM lista_reproduccion WHERE etiquetas CONTAINS 'guitarra';
```

- 15) Es posible eliminar registros particulares con el siguiente comando:

```
cqlsh:demo_cql_music> DELETE FROM lista_reproduccion WHERE id = 62c36092-82a1-3a00-93d1-46196ee77204 AND nro_cancion = 1;
```

- 16) O todos los registros de la lista de reproducción creada, con el siguiente comando:

```
cqlsh:demo_cql_music> DELETE FROM lista_reproduccion WHERE id = 62c36092-82a1-3a00-93d1-46196ee77204;
```

- 17) Para eliminar el Column Family, utilice la siguiente sentencia:

```
cqlsh:demo_cql_music> DROP TABLE lista_reproduccion;
```

- 18) Para finalizar, elimine el Keyspace:

```
cqlsh:demo_cql_music> DROP KEYSPACE demo_cql_music;
```